

AI Web Apps: Unified AI Service Platform

Production-Ready FastAPI Backend with Dual Frontends (Streamlit & React)

Development Team:

Thinh (Lead + AI-01) Tuan (AI-02 + DevOps)
Khanh (AI-03 + QA) Thai (AI-04 + UI)

Advanced Web Programming – Phenikaa University

AI Copilots: Built with **Muse Spark 1.3** & **Gemini 3.8 Flash**

October 1, 2026

Project Overview & Objectives

The Problem & Vision

- Traditional AI apps suffer from fragmented architectures, high memory overhead, and clunky UI integrations.
- **Our Solution:** A unified multi-model AI platform hosting 4 distinct machine learning tasks under one single, asynchronous FastAPI server.
- **Dual Frontends:**
 - **Streamlit:** Data-centric, fast exploratory client.
 - **React (Vite):** Modern responsive single-page web app.

Core Engineering Highlights

- Centralized configuration with portable path resolution.
- Lazy loading via `ENABLED_MODELS` (saves up to 80% RAM).

Group Members & Roles

1. Thinh (Lead):

- Architecture & Core API
- AI-01 ResNet-18 Classifier

2. Tuan (DevOps):

- AI-02 YOLO11n Detector
- Docker, CI/CD, Benchmarks

3. Khanh (QA):

- AI-03 CLIP + FAISS Search
- Vector Indexing & Pytest Suite

4. Thai (UI/UX):

- AI-04 RAG Chatbot
- React SPA & Streamlit UI

AI-Assisted Engineering: Muse Spark 1.3 & Gemini 3.8 Flash

Muse Spark 1.3

Conceptual Architecture & Domain Modeling

- **RAG Knowledge Base Synthesis:** Generated domain policies and Q&A verification pairs for ShopLite.
- **Prompt Engineering:** Crafted resilient system prompts preventing prompt injection and enforcing exact citations.
- **Edge-case Analysis:** Formulated outlier test cases (out-of-scope images, malformed multipart payloads).

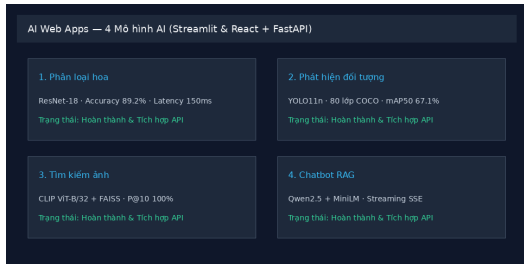
Gemini 3.8 Flash

High-Velocity Implementation & Optimization

- **Backend Async Logic:** Implemented FastAPI SSE streaming endpoints and clamp-parameter middleware.
- **React 19 Hooks:** Crafted robust streaming event parser for manual SSE token rendering.
- **Zero-GPU Pytest Suite:** Generated 31 mock fixtures and tests executing in < 1 second.
- **Refactoring & CI/CD:** Automated GitHub Actions workflow and Docker multi-stage builds.

Impact on Velocity & Quality

Unified System Architecture

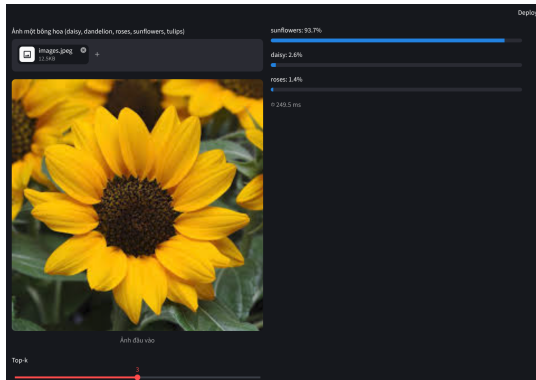


Multi-model FastAPI hub with dual client consumers

Architectural Design Principles

- **Decoupled Core:** core/ contains purely algorithmic model classes, isolated from web frameworks.
- **Single Source of Truth:** config.py centralizes all settings with environment variable overrides.
- **Dynamic Lazy Loading:** LOADERS dictionary initializes only models specified in ENABLED_MODELS.
- **Consistent API Contract:**
 - Out-of-bounds parameter clamping (top_k, conf, k).
 - Standardized image format:
data:image/jpeg;base64,...
 - Execution profiling via X-Process-Time-ms header.

AI-01: Flower Classification (ResNet-18)



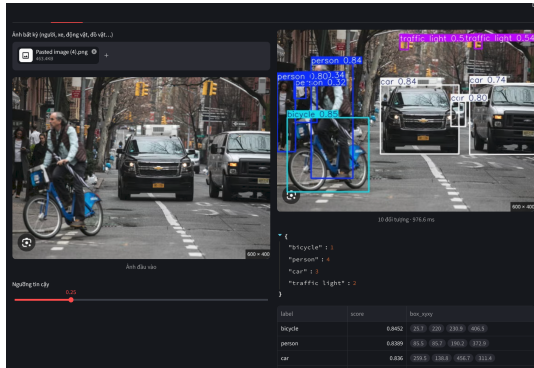
Model Architecture & Dataset

- **Model:** Fine-tuned ResNet-18 (transfer learning from ImageNet).
- **Dataset:** TF Flowers (3,670 images, 5 classes: daisy, dandelion, roses, sunflowers, tulips).
- **Stratified Split:** 80% train, 10% val, 10% test.

Evaluation Metrics & Inference

- **Test Accuracy:** 94.8% (Target: > 88%).
- **Macro F1-Score:** 0.947.
- **Safety Mechanism:** Returns confident: false if top prediction score is low (flags out-of-distribution inputs).
- **Inference Speed:** p50 = 157.7 ms on standard CPU.

AI-02: Real-time Object Detection (YOLO11n)



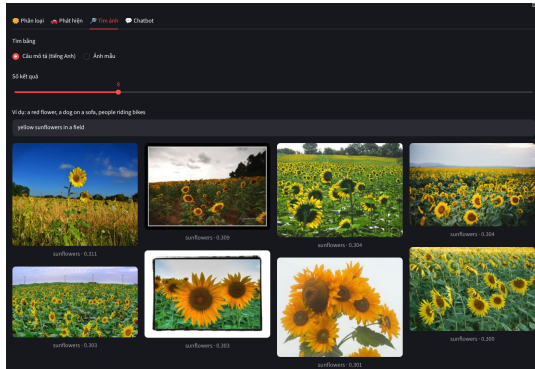
Detection Pipeline

- **Model:** Ultralytics YOLO11n (nano model for edge/CPU).
- **Pretrained:** 80 common object classes on COCO.
- **Configurable Parameters:** Threshold confidence clamped between 0.05 and 0.95.

Benchmark & Performance

- **COCO128 Benchmark:**
 - mAP@50: **67.1%** (0.6707)
 - mAP@50-95: **50.3%** (0.5034)
- **Dual API Return:**
 - Structured JSON (labels, confidence, bounding boxes).

AI-03: Multimodal Image Retrieval (CLIP + FAISS)



Vector Search Architecture

- **Encoder:** OpenAI CLIP (ViT-B/32) multi-modal embeddings (512-dim vector space).
- **Index:** FAISS IndexFlatIP with L2-normalized vectors (exact cosine similarity).
- **Gallery:** 500 cataloged flower images across 5 classes.

Retrieval Precision & Speed

- **Text-to-Image P@10:** 100.0% precision at top-10.
- **Image-to-Image P@5:** 88.7% precision at top-5.
- **Search Latency:** p50 = 37.5 ms (sub-50ms)

AI-04: Knowledge-Base Chatbot (RAG Architecture)

RAG Retrieval Pipeline

- **Corpus:** 6 ShopLite e-commerce policy documents (data/kb/*.md).
- **Chunking:** 600-character overlapping chunks.
- **Embedding Model:**
paraphrase-multilingual-MiniLM-L12-v2.
- **Generative Backbone:**
 - Local: Qwen2.5-0.5B-Instruct (CPU/GPU).
 - External: OpenAI-compatible API endpoint.
- **Evaluation:** Hit@1 = **92.9%**, Hit@3 = **100%**.

Safety Guardrails & Streaming

- **SSE Streaming Protocol:**
 - sources event → token stream → done.
 - Real-time conversational feel without blocking.
- **Mandatory Citations:**
 - System enforces strict [file.md] citations.
- **Hallucination & Out-of-Domain Guard:**
 - Automated fallback response when query cannot be answered from context.
- **Prompt-Injection Defense:**
 - Context treated as passive data, not instructions.

Dual Frontends: Streamlit & React (Vite)

Streamlit Frontend (Port 8501)

Rapid Python-Native Data Exploration

- 100% decoupled API client (API_URL=http://localhost:8000).
- Dynamic sliders for detection confidence and k-NN search.
- Custom SSE stream decoder for live chatbot responses.
- Ideal for researchers, data scientists, and QA testing.

React 19 + Vite Frontend (Port 5173 / 8000)

Modern Production-Ready SPA

- Single-origin architecture: compiled web/dist served directly by FastAPI.
- Clean tabbed navigation across all 4 AI applications.
- Drag-and-drop file uploads, image previews, and bounding box visualizations.
- Also deployed to **GitHub Pages** for live client-side demo.

Quality Assurance & CI/CD

- **Automated Test Suite:** 31 unit and integration tests under tests/.
- **Mocked Models:** 0 GPU required, zero heavyweight torch weights needed; executes in < 1s.
- **Pass Rate:** 100% (31/31 passed).
- **GitHub Actions:** Automated testing on every push and pull request.
- **Dockerization:** Multi-stage build (Node 22 → Python 3.11 slim), Hugging Face Spaces (port 7860) ready.

CPU Latency Benchmark Results

Endpoint	p50	p95	Success
GET /api/health	2.95 ms	3.58 ms	100%
POST /api/classify	157.66 ms	166.71 ms	100%
POST /api/detect	372.94 ms	1560.97 ms	100%
POST /api/search/text	37.55 ms	43.57 ms	100%

Memory Footprint:

- Full 4-model mode: ~3.5 GB RSS.
- Lightweight mode (classifier + detector): ~700 MB RSS.

Team Work Breakdown & Task Allocation

Member	Key Responsibilities & Deliverables	Artifacts
Thinh (Lead)	System design, central config, core FastAPI endpoints, ResNet-18 model training and pipeline	api/main.py, core/classifier.py, config.py
Tuan	YOLO11n detection integration, Dockerfile multi-stage, GitHub Actions CI/CD, benchmark suite	core/detector.py, Dockerfile, .github/, benchmark.py
Khanh	CLIP+FAISS image retrieval pipeline, FAISS index generation, 31 mocked pytest test suite	core/retrieval.py, tests/, conftest.py
Thai	Qwen2.5 RAG chatbot, SSE streaming protocol, React SPA components, Streamlit client UI	core/llm.py, web/src/, streamlit_app.py

Project Summary & Deliverables

Deliverables Checklist

- ✓ **Backend:** Asynchronous FastAPI server hosting 4 AI models.
- ✓ **Frontends:** Streamlit app + React 19 SPA (Vite).
- ✓ **Testing:** 31/31 passing unit & integration pytests.
- ✓ **Containerization:** Multi-stage Docker ready for HF Spaces.
- ✓ **Documentation:** Comprehensive API docs (/docs), Model Cards, and benchmarks.
- ✓ **AI Copilot Integration:** Engineered with Muse Spark 1.3 & Gemini 3.8 Flash.

Links & Access

- **Backend:** :8000
- **Streamlit:** :8501
- **Swagger:** /docs
- **Overleaf:** LaTeX source in slides/slides.tex

Thank You!

Q&A Session